# Software Requirements Specification

## for

# Travelly

**Version 1.0**

## Prepared by


**Group #17** **Group Name: 10 Bit**

| | | |
|---|---|---|
| Ayush Jha | 230258 | ayushj23@iitk.ac.in |
| Khushi Khandelwal | 230561 | khushik23@iitk.ac.in |
| Pranjali Singh | 220796 | pranjalis22@iitk.ac.in |
| Mohd Sarim Ali | 230657 | sarimali23@iitk.ac.in |
| Raghav Madan | 220853 | raghavm22@iitk.ac.in |
| Rigved Koli | 230855 | brigved23@iitk.ac.in |
| Ronit Kumar | 230875 | ronitk23@iitk.ac.in |
| Rushabh Pandya | 230732 | rushabhbp23@iitk.ac.in |
| Sanket Agarwal | 230920 | sanketag23@iitk.ac.in |
| Tanmay Roy | 231082 | tanmayroy23@iitk.ac.in |


**Course:** CS253

**Mentor TA:** *V Arvind*

**Date:** 24/01/2026

# Contents

**Contents** ..... 2

**Revisions** ..... 3

**1 Introduction** ..... 4

1.1 Product Scope ..... 4
1.2 Intended Audience and Document Overview ..... 4
1.3 Definitions, Acronyms and Abbreviations ..... 5
1.4 Document Conventions ..... 5
1.5 References and Acknowledgments ..... 6

**2 Overall Description** ..... 7

2.1 Product Overview ..... 7
2.2 Product Functionality ..... 8
2.3 Design and Implementation Constraints ..... 8
2.4 Assumptions and Dependencies ..... 9

**3 Specific Requirements** ..... 10

3.1 External Interface Requirements ..... 10
3.2 Functional Requirements ..... 15
3.3 Use Case Model ..... 19

**4 Other Non-functional Requirements** ..... 32

4.1 Performance Requirements ..... 32
4.2 Safety and Security Requirements ..... 32
4.3 Software Quality Attributes ..... 33

**5 Other Requirements** ..... 35

5.1 Authentication and Verification Requirements ..... 35
5.2 Legal, Privacy and Compliance Requirements ..... 35
5.3 Payments and Financial Safety Requirements ..... 35
5.4 Database and Backup Requirements ..... 35
5.5 Third-Party Service Dependency Requirements ..... 36
5.6 Data Retention and Cleanup Requirements ..... 36

**Appendix A – Data Dictionary** ..... 37

**Appendix B - Group Log** ..... 39

# Revisions

| Version | Primary Author(s) | Description of Version | Date Completed |
|---|---|---|---|
| Version 1.0 | Ayush Jha<br>Khushi Khandelwal<br>Pranjali Singh<br>Mohd Sarim Ali<br>Raghav Madan<br>Rigved Koli<br>Ronit Kumar<br>Rushabh Pandya<br>Sanket Agarwal<br>Tanmay Roy | First Version of the Software Requirement Specification (SRS) Document | 24/01/26 |

# 1 Introduction

## 1.1 Product Scope

Travelly is a trip management application designed to simplify the planning, coordination, and financial management of group travel. The application provides a centralized platform where users can create trip groups, manage shared expenses, track individual and group balances, and store trip-related photos and documents. The main purpose of the application is to reduce confusion and inefficiencies commonly faced during group trips by offering structured tools for coordination and financial management.

A primary objective of Travelly is to ensure transparency and fairness in expense handling. The application incorporates structured expense-splitting mechanisms and flow-based debt simplification algorithms to minimize redundant transactions while maintaining accurate financial relationships among group members. Additionally, the flexible group joining mechanism allows expenses to be recorded even when all participants have not yet joined the application, ensuring continuity and consistency throughout the duration of the trip.

In addition to expense management, Travelly enhances the overall travel experience by enabling secure document storage, photo sharing, customizable group settings, and seamless settlement through UPI-based payment redirection. By combining financial clarity, organized trip data, and ease of use, the application aims to provide a seamless, efficient, and stress-free experience for individuals traveling together in groups.

## 1.2 Intended Audience and Document Overview

This Software Requirements Specification (SRS) document is mainly intended for the people connected to the project, namely, the course instructor, our TA, our team, and the team that intends to beta-test the application.The document may also be useful to end users who wish to understand the scope, features, and intended behavior of the application prior to or during its use.

The document is divided into the following sections:

**Section 1:**
This section introduces the purpose and scope of the Software Requirements Specification (SRS) document. It defines key terminology, abbreviations, and essential background information required to understand the system. This section also provides a high-level overview of the entire document. Users who are already familiar with the terminology may skip this section.

**Section 2:**
This section provides a detailed overall description of the application, including its general overview, core functionality, design and implementation constraints, as well as assumptions and dependencies. It helps readers understand how the system is expected to operate within its environment. All users are encouraged to read this section to gain a comprehensive understanding of the application.

**Section 3:**
This section presents the specific requirements of the system. It includes external interface requirements, functional requirements, and the use case model. The complete user experience is

described through various use cases, explaining how users interact with the application to accomplish their tasks. End users and developers should carefully review this section to fully understand the operational requirements of the system.

**Section 4:**
This section outlines the non-functional requirements of the application, including performance requirements, safety and security requirements, and software quality attributes. These requirements ensure reliability, usability, and overall system effectiveness. This section is critical for all users to read in order to understand the constraints and quality expectations of the application.

## 1.3 Definitions, Acronyms and Abbreviations

- **API** – Application Programming Interface
- **INR** – Indian Rupee
- **OTP** – One-Time Password
- **SRS** – Software Requirements Specification
- **UPI** – Unified Payments Interface
- **UI** – User Interface
- **UX** – User Experience
- **DPDPA** – Digital Personal Data Protection Act
- **OS** – Operating System
- **ID** – Identifier
- **URL** – Uniform Resource Locator
- **DB** – Database
- **FR** – Functional Requirement
- **UAT** – User Acceptance Testing

## 1.4 Document Conventions

**Formatting Conventions:**

- The document follows IEEE SRS formatting guidelines.
- The main text is written in Cambria Math, font size 11, and is single-spaced.
- Section and subsection headings are in Cambria Math, font size 14, to clearly differentiate them from body text.
- Margins are maintained at 1 inch on all sides.
- Italics are used for comments, notes, or explanatory text within the document.

**Naming Conventions:**

- All variables, functions, and system components mentioned in the document follow camelCase or descriptive naming for clarity.
- Abbreviations and acronyms are defined in Section 1.3 and used consistently throughout the document.
- Feature names, module titles, and tab names reflect their exact naming in the application to avoid confusion during development or testing.

**General Conventions:**

- Bullet points are used for clarity where multiple items or requirements are listed.
- Numbered lists are used for sequences, steps, or hierarchical requirements.
- Terminology specific to the Travelly app (e.g., flow-based algorithm, trip group, simplified debts) is used consistently and defined in the glossary when necessary.

## 1.5 References and Acknowledgments

### 1.5.1 References:

Style guide (for this document): IEEE Software Requirements Specifications Guide

### 1.5.2 Acknowledgement:

We are highly indebted to Professor Indranil Saha for his excellent lectures, which largely provided the basis for our system design, and for diligently resolving all our queries regarding various functional and organizational aspects of the software. We are also obliged to our Teaching Assistant, V Arvind, for his timely assistance whenever required. We used Canva for creating the various diagrams of the use cases and Figma for the various UI designs that have been used to present the basic ideas of our application effectively.

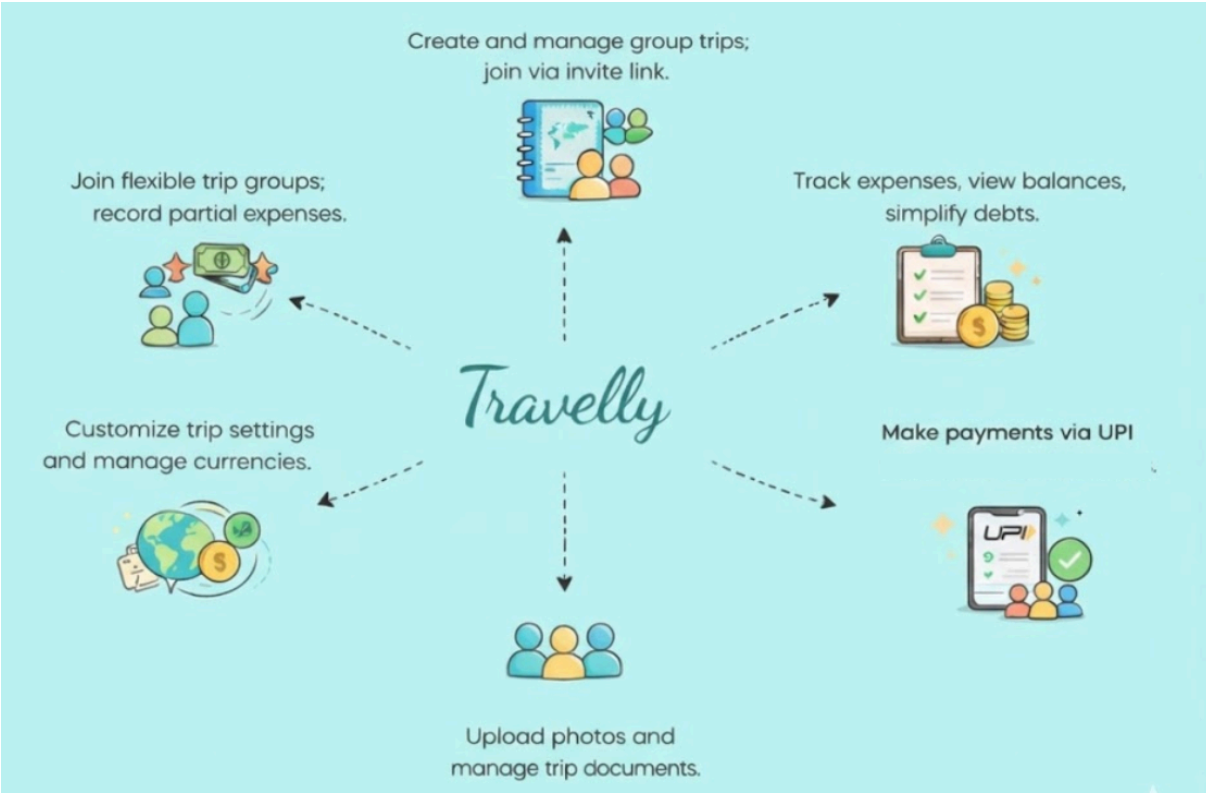
# 2 Overall Description

## 2.1 Product Overview

Travelly is a standalone trip management application designed to simplify group travel by addressing challenges related to expense tracking, settlement, and trip-related information management. The system is inspired by limitations in existing expense-sharing platforms and introduces a flexible group participation model that allows expenses to be recorded even when all members have not yet joined the application.

The application follows a client–server architecture, where the mobile app acts as the primary interface for users, and the backend manages business logic, data processing, and persistent storage. It supports multiple trip groups, each maintaining its own expenses, participants, settings, photos, and documents.

Travelly integrates authentication services, UPI-based payment redirection, and cloud storage into a unified platform. Its modular architecture ensures scalability and enables future feature expansion without affecting existing system functionality.



## 2.2 Product Functionality

- **User Registration and Authentication**
  The system provides user registration and authentication using email address or phone number with OTP-based verification. It also supports password-based login, Google Sign-In, password recovery, and secure account management.

- **User Profile and Preferences Management**
  The system allows users to view and update personal profile information, configure application theme preferences, manage notification settings, access application information, and perform account-related actions such as logout or profile deletion.

- **Group Creation and Joining**
  The system enables users to create trip groups, add participants in advance, generate a common invite link, and allow users to join groups later by selecting or adding their name, ensuring flexible group participation.

- **Expense Logging and Management**
  The system allows users to record group expenses using multiple splitting methods, including equal splits, selected participants, and custom amounts. A complete and chronological expense history is maintained for transparency.

- **Group Balances and Debt Simplification**
  The system computes individual and group balances, displays amounts owed or receivable, and generates reimbursement transactions. It supports debt simplification to reduce the total number of transactions.

- **UPI-Based Settlement**
  The system supports settlement of outstanding balances through UPI-based payment redirection as well as manual settlement confirmation, with balances updated upon user confirmation.

- **Photos and Documents Management**
  The system provides trip-wise storage for photos and documents, allowing users to upload, view, download, and manage files associated with a trip securely across devices.

- **Route Optimization and Visit Planning**
  The system generates an optimized visiting order for selected locations within a trip by minimizing the total travel distance.

## 2.3 Design and Implementation Constraints

The development of the Travelly application is constrained by mobile device limitations, including processing power, memory availability, and battery consumption, requiring efficient resource usage across a wide range of smartphones. The application must support both Android and iOS platforms using a unified cross-platform framework. It depends on external services such as authentication providers, location and map services, and UPI payment applications, which impose constraints related to availability, rate limits, and platform-specific policies.

Additionally, the system must be developed using predefined technologies and frameworks to ensure scalability and maintainability. All communication between the application and backend services must use secure communication protocols. Standard coding practices and design conventions must be followed to ensure maintainability and scalability.

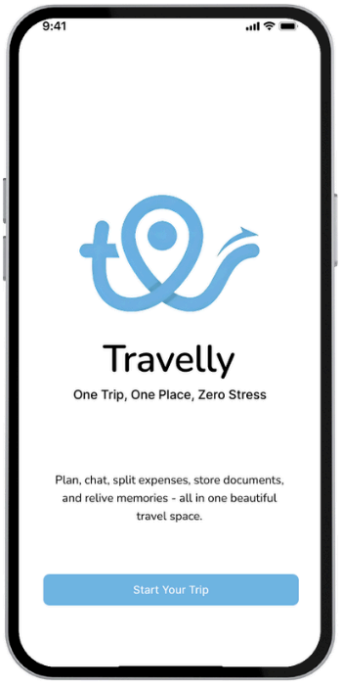
## 2.4 Assumptions and Dependencies

- Users are assumed to have a stable internet connectivity for features such as group synchronization, settlements, expense updates, and media access.
- Continued availability and support of third-party services such as Google authentication, location services, and UPI payment applications is assumed.
- External APIs used for location, maps, or travel-related information are assumed to maintain backward compatibility in data formats and access policies.
- The system depends on mobile OS support for notifications, background processing, and deep linking to external payment applications.
- Users are assumed to grant necessary permissions such as location and notification access for full functionality.
- The backend services and database infrastructure are assumed to be available with minimal downtime to ensure data consistency across devices.
- The design assumes reuse of standard libraries, frameworks, and components provided by Flutter, Node.js, and SQL-based databases.
- Users are assumed to act honestly when confirming or manually marking settlements as paid.

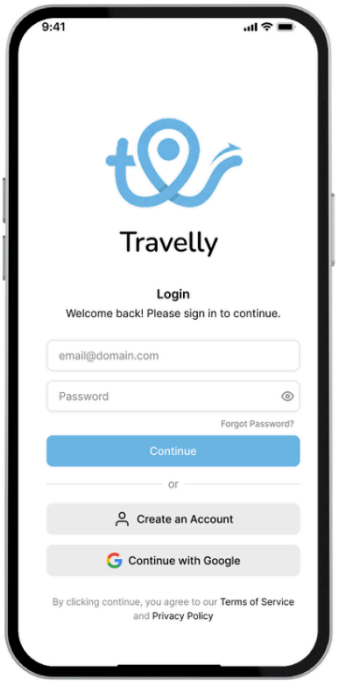
# 3 Specific Requirements

## 3.1 External Interface Requirements

### 3.1.1 User Interfaces




Onboarding Screen



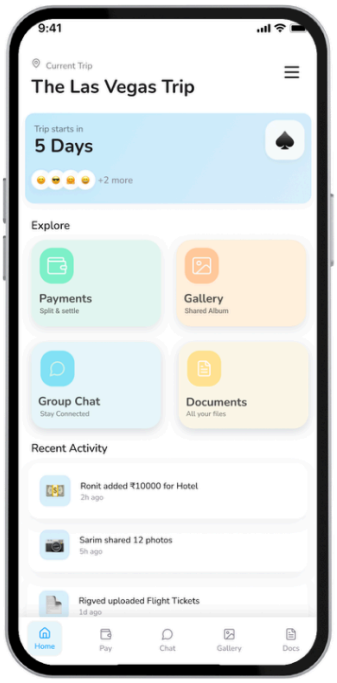

Login Screen

- **Onboarding Screen:** The onboarding screen introduces users to the Travelly application and briefly explains its purpose and key features. It provides a clean and welcoming interface for first-time users. This screen helps users understand how Travelly simplifies trip planning. It guides them toward starting their journey within the app.

- **Login Screen:** The login screen enables registered users to securely access their Travelly account. Users can sign in using their email or phone number with a password or choose Google sign-in. It also includes options for account creation and password recovery. This ensures a smooth and secure authentication experience.



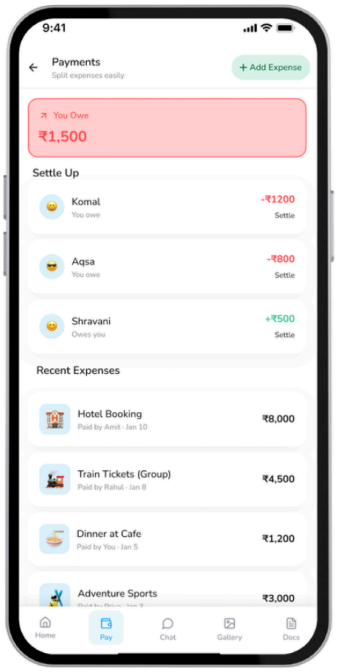

Dashboard




Trip Dashboard

- **Dashboard:** Displays all trips created or joined by the user in a structured and visually intuitive layout. It allows quick access to ongoing and past trips. Users can easily select a trip to continue planning or review previous activities. This screen acts as the central hub of the application.

- **Trip Dashboard:** Provides a comprehensive overview of the selected trip. It offers quick access to key modules such as expenses, gallery, and documents. Recent activities are displayed to keep users updated. This screen helps users manage all trip-related information from one place.



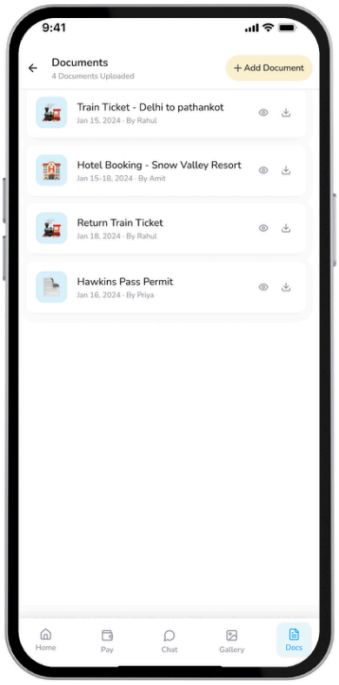

Transactions




Gallery

- **Transactions:** Shows all trip-related expenses, balances, and settlement details in a clear and organized manner. It helps users track who paid, who owes, and the total spending. The screen supports transparent expense management throughout the trip. Users can easily initiate or review settlements from here.

- **Gallery:** Allows users to upload, view, and manage trip photos in a shared visual space. All participants can access memories captured during the trip. Images are displayed in a grid layout for easy browsing. This feature helps preserve and relive trip experiences collectively.




Documents

- **Documents:** Enables users to upload, view, download, and manage important trip-related documents such as tickets and reservations. All documents are stored in a centralized and organized manner, ensuring easy access

### Authentication Interfaces

#### Login Interface

- The system shall allow users to log in using their registered email address or phone number and password.
- Upon successful authentication, the system shall redirect the user to the home/dashboard interface.

#### Sign-Up Interface

- The system shall allow new users to register by providing a valid email address or phone number.
- The system shall verify the provided contact details using OTP-based verification.
- Upon successful verification, the system shall allow the user to create a password and complete registration.
- After successful registration, the system shall redirect the user to the login interface.

#### Forgot Password Interface

- The system shall allow users to initiate password recovery by entering their registered email address or phone number.
- The system shall verify the user using OTP-based authentication.
- Upon successful verification, the system shall allow the user to reset the password and redirect them to the login interface.

#### User Profile Interface

#### Profile Information

- The system shall display user profile information, including profile photo, name, email address, and phone number.
- The system shall allow users to edit and update their own profile information.
- Profile modification capabilities shall be restricted to the authenticated user's own profile.

### 3.1.2 Hardware Interfaces

The Travelly application operates on smartphones and interfaces with the following hardware components:

- **Touchscreen Interface**
  - The app will rely on touch gestures for navigation, adding expenses, settling balances, and managing groups.
  - The system shall support standard touchscreen interactions such as taps, long-press actions, swipes, and pinch-to-zoom gestures for navigation and content interaction.
- **Device Storage**
  - The system shall access device storage for caching application data, expense records, and downloaded photos or documents.
  - Support for external storage is dependent on the underlying mobile operating system.
- **Camera Interface**
  - The system shall allow users to capture and upload photos or documents using the device camera through standard operating system camera services.
  - Utilizes the device camera hardware through OS camera services.
- **Network Interface**
  - The system shall require Wi-Fi or cellular network connectivity for authentication, data synchronization, settlements, and media upload/download operations.
  - Compatible with standard smartphone Wi-Fi and cellular hardware.

- **Location Services**
  - The system may use device location services to determine default currency settings or provide location-based features, subject to user permission.

- **Biometric Authentication**
  - The system may support biometric authentication (e.g., fingerprint or facial recognition) for application access or confirmation of sensitive actions, subject to device capability and user consent.

### 3.1.3 Software Interfaces

- The application must be compatible with modern Android and iOS operating systems, ensuring smooth functionality across a wide range of devices. Specifically:
  - Android: The application must support devices running Android 8.0 (API Level 26) and above.
  - iOS: The application must support devices running iOS 13 and above.
  - The application must function seamlessly across various screen sizes and resolutions to ensure consistent usability across smartphones and tablets.

- The system shall integrate with external services and APIs, including:

- OTP generation and verification services for email and phone authentication

- Optional third-party authentication services (e.g., Google Sign-In)

- UPI intent or deep-linking mechanisms for initiating payment settlements

- Push notification services for settlement request/reminders, group updates, and system alerts

- The system shall use backend services and a database to store and manage data related to users, groups, expenses, balances, settlements, and group settings.

- The system shall use a file storage service to upload, store, and retrieve photos and documents associated with trip groups, with metadata linking files to corresponding trips.

- The application may integrate with maps/location services for place suggestions and default currency for settlement .

## 3.2 Functional Requirements

**FR-1: User Registration**

- The system shall allow users to register using a valid email address or phone number. During registration, the system shall verify the provided email address or phone number

through a One-Time Password (OTP). Upon successful verification, users shall be required to create a secure password for subsequent authentication.

**FR-2: User Login**

- The system shall allow registered users to log in using their email address or phone number along with a password.
- In addition, the system shall support alternative authentication methods, including Google Sign-In and OTP-based login, where an OTP is sent to the user's registered email address or phone number for verification.

**FR-3: Forgot Password & Credential Recovery**

- The system shall provide a password recovery mechanism that allows users to reset their login password by verifying their identity through an OTP sent to the registered email address or phone number.

**FR-4: View & Manage User Profile**

- The system shall allow users to view and update their personal profile information, including name, email address, and phone number. Any change to the registered email address or phone number shall require verification through an OTP sent to the updated contact detail.
- The system shall allow users to manage application preferences, including selecting a light theme, dark theme, or synchronizing the application theme with the device's system settings.
- The system shall provide a global notification preference that enables or disables alerts for key events such as settlement requests, payment reminders, group updates, and critical security-related activities.
- The system shall allow users to securely log out of the application and permanently delete their user profile, subject to confirmation.
- The system shall provide an "About Travelly" section that displays information about the application, its features, and the development team.

**Creating and Joining a Group**

- The system shall allow users to create a new group or join an existing group by clicking on the add group option in the groups tab.

**FR-5: Create a new group:**

- The system shall allow users to create a new group by providing a group title and optionally setting a group photo. If no group photo is provided, the system shall assign a default image.
- The system shall automatically select a default currency for the group based on the user's location, if available. If location access is not granted, the system shall use INR as the default currency. Users shall have the option to change the selected currency during group creation.
- The system shall allow the creator to add the names of group participants in advance. Upon successful group creation, the system shall generate a single, shareable invite link that can be used by participants to join the group.

- The system shall allow additional participants to be added later through add participants option in the group settings and the user can add the name of the participant and share the same invite link to the participant.

**FR-6: Join an existing group:**

- The system shall allow users to join an existing group using a valid invite link. The user can either directly click on the invite link, which will be redirected to the Travelly application or can manually paste the invite link after clicking on join an existing group option.
- Upon joining, the system shall prompt the user to select their name from the list of pre-added participants. If the user's name is not present, the system shall allow the user to join as a new participant by entering their name.
- Participants who join a group later shall be able to view all existing group information, including expenses (including those they were part of), balances, photos, and documents relevant to the trip.

**FR-7: Delete a group:**

- The system shall allow users to delete a group.
- Upon deletion, the system shall revoke the group invite link and restrict access to the group's data for all members.
- The system shall require confirmation before permanently deleting a group to prevent accidental data loss.

**FR-8: Group Expense Management**

- The system shall allow users to record expenses within a group by specifying an expense title, amount , payer, and date of payment. By default, the system shall associate each expense with the currency selected during group creation. Users shall be allowed to record expenses in a different currency when required.
- The system shall support multiple expense-splitting methods, including:
- Equal split among all group members,

- Equal split among selected participants, and

- Custom split amounts assigned to individual group members.

- The system shall allow users to record direct monetary transfers between group members by specifying the transfer title, amount, sender, and receiver.
- The system shall display a complete expense history for the group, including total group expenditure and individual spending, with expenses ordered chronologically.
- Participants who join a group at a later time shall be able to view all previously recorded expenses to ensure transparency and accurate settlement throughout the trip.

**FR 9: Group Balances and Debt Simplification**

- The system shall compute and display the net balance for each member of a group, indicating the total amount owed or to be received by the member. A positive balance shall indicate that the member is owed money, while a negative balance shall indicate that the member owes money.

- The system shall provide an overview of all group balances on clicking "View Reimbursements", and generate a list of reimbursement transactions required to settle the outstanding balances.
- For each reimbursement transaction:
- If a user is owed an amount by another user, the system shall allow two options: Request and Mark as Paid. The request option sends a payment reminder to the debtor while the mark as paid option marks a transaction as settled for manual payments.
- If a user owes an amount to another user, the system shall allow two options: Pay and Mark as Paid. The Pay option redirects to an installed UPI application and shows a popup on return: "Did the payment go through?" (Yes/No) and the system shall settle the debt accordingly by updating balances and records in the database. The system shall allow payment initiation only when the payee has a valid settlement method saved in their profile. The Mark as Paid option can be used if user A has paid user B manually.
- By default, the system shall minimize the total number of required reimbursement transactions while strictly preserving the original debtor-creditor relationship i.e no one owes a person they didn't owe before. The optimization algorithm must ensure that the final list of payees for any user is a subset of the original list of creditors.
- The system shall provide a "Simplify Debts" option that, when enabled, minimizes the total number of reimbursement transactions required to settle the group balances. When this option is enabled, the original debtor-creditor relationships may not be preserved and the number of settlement transactions will be lesser than the transactions shown when Simplify Debts is OFF.

### FR 10: UPI-Based Settlement Integration

- The system shall allow users to save and update their UPI ID within their profile or application settings for use during settlement.
- The system shall enable users to settle outstanding balances they owe by redirecting them to an installed UPI application (GPay / PhonePe / Paytm) , with the payee UPI ID, payable amount, and an optional transaction note pre-filled.
- Upon returning from the UPI application, the system shall prompt the user to confirm whether the payment was completed successfully and shall update balances and settlement records accordingly.
- The system shall allow users to manually mark a settlement as completed for non-UPI payments, subject to user confirmation, and shall record such settlements with the payment method identified as manual.

### FR 11: Photos and Documents Manager

- The system shall provide a trip-wise media management feature that allows users to upload, view, download, and delete photos and documents associated with a specific trip using an intuitive grid (photos) and list (documents) interface.
- The system shall organize photos and documents in a structured manner to ensure clear separation between different trips and ease of navigation for users.
- The feature will be implemented using the same component-based UI pattern as other modules in the app, ensuring consistency and easy scalability across the trip management system.

- The system shall ensure that uploaded media is securely stored through cloud-based object storage, where files are stored securely and referenced via trip-specific IDs in the database, enabling access across multiple user devices belonging to the same trip group.
- The system shall enforce constraints on media uploads, including file size limits and supported formats, and shall apply performance optimizations to ensure efficient storage usage and responsive media access.
- The system shall ensure that uploaded media is securely stored and accessible across multiple user devices belonging to the same trip group.

**FR 12: Route Optimization and Visit Planning**

- The system shall allow users to specify a starting location, an ending location, and a set of intermediate locations to be visited during a trip.
- Based on the provided locations, the system shall compute and display an optimized visiting order that covers all selected locations while minimizing the total travel distance.

## 3.3 Use Case Model

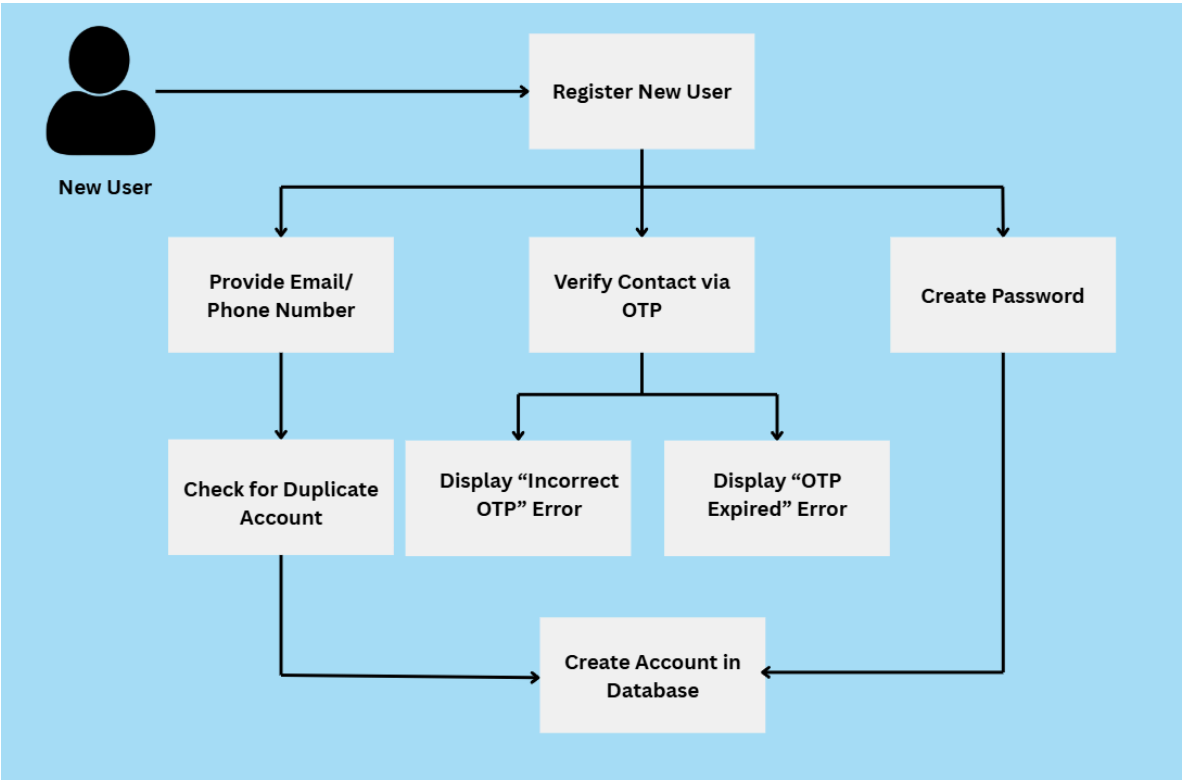
The use case model describes the complete interaction between users and the Travelly system. Each use case represents a specific goal that a user aims to achieve while interacting with the application. These use cases collectively capture the functional behavior of the system and define how the system responds to user actions under various conditions.

The following actors are involved in the use case model:

- **User** - A registered user of the Travelly application. Primary actor who interacts with all core functionalities including registration, group management, expense logging, and settlement.
- **Payment Gateway** - External UPI payment applications (Google Pay, PhonePe, Paytm) that handle financial transactions. Participates in the settlement process by redirecting users for payment confirmation.
- **System** - Backend services handling computation, storage, validation, authentication, expense calculations, balance management, and database operations.
- **Location Service** - External API providing place and distance information

A comprehensive use case diagram encapsulating all actors and system interactions shall be provided.

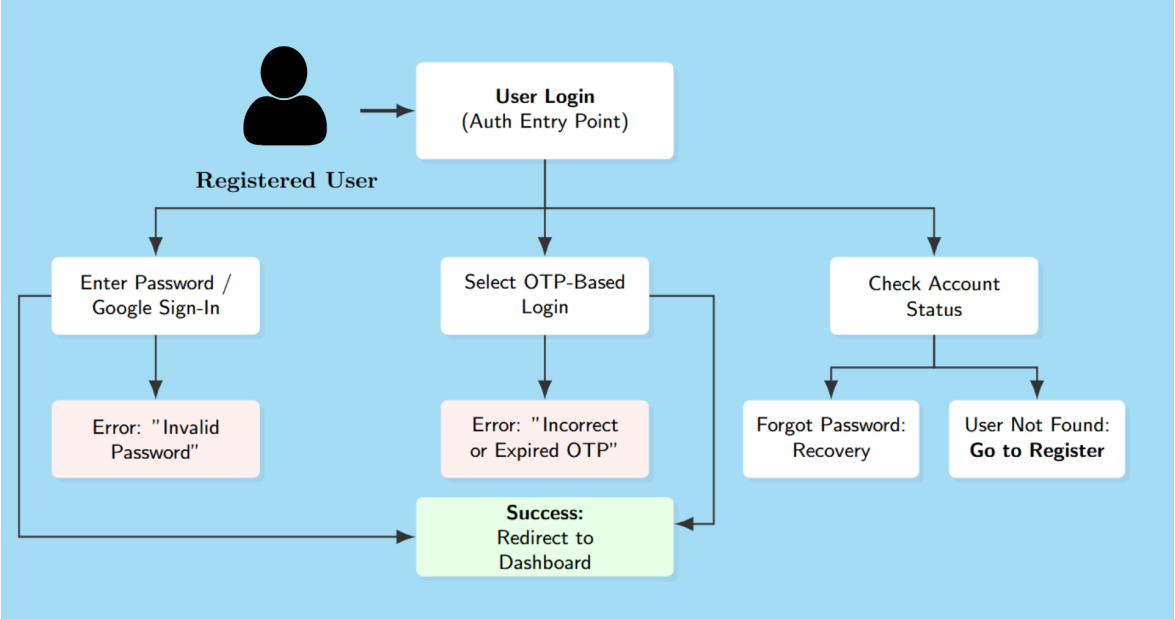
### Use Case #1: User Registration (U1)




| | |
|---|---|
| Authors | This use case was written by Ayush Jha and Khushi Khandelwal. |
| Purpose | To allow a new user to create an account on the Travelly application. |
| Requirements Traceability | FR-1 |
| Priority | High - Mandatory for accessing the system. |
| Preconditions | The user must not already be registered in the system. |
| Post conditions | • A new user account is successfully created and stored in the system database.<br>• The user's email address or phone number is verified and linked to the account. |

| | |
|---|---|
| Actors | New User, System |
| Exceptions | • Duplicate Registration: If the provided email address or phone number is already registered, the system shall display the message: *"Account already exists. Please log in or reset your password."*<br>• OTP Verification Failure:<br>1. If the user enters an incorrect OTP, the system shall display: *"Incorrect OTP entered. Please try again."*<br>2. If the OTP validity period expires, the system shall display: *"OTP expired. Please request a new OTP."*<br><br>• OTP Delivery Failure: If the OTP cannot be delivered due to network or service issues, the system shall prompt the user to retry. |
| Includes | None |
| Notes/Issues | • The system shall send an OTP to the email address or phone number provided during registration for verification.<br>• OTPs shall have a predefined validity period (e.g., 10 minutes) to enhance security.<br>• Successful OTP verification is mandatory before allowing the user to set a password and complete registration. |

**Use Case #2: User Login (U2)**




| | |
|---|---|
| Authors | This use case was written by Tanmay Roy and Rigved Koli. |
| Purpose | To authenticate a registered user and allow access to the Travelly application. |
| Requirements Traceability | FR-2, FR-3 |
| Priority | High - Mandatory for accessing application features. |
| Preconditions | The user must be registered in the system. |
| Post conditions | The user is successfully authenticated and redirected to the home/dashboard screen. |
| Actors | User, System |
| Exceptions | • Unregistered User: If the provided email address or phone number is not registered, the system shall display: *"User not found. Please register before logging in."*<br>• Invalid Credentials: If the user enters an incorrect password, the system shall display: *"Invalid password. Please try again."*<br>• Password Recovery Triggered: If the user selects the "Forgot Password" option, the system shall initiate the password recovery process by sending an OTP to the registered |

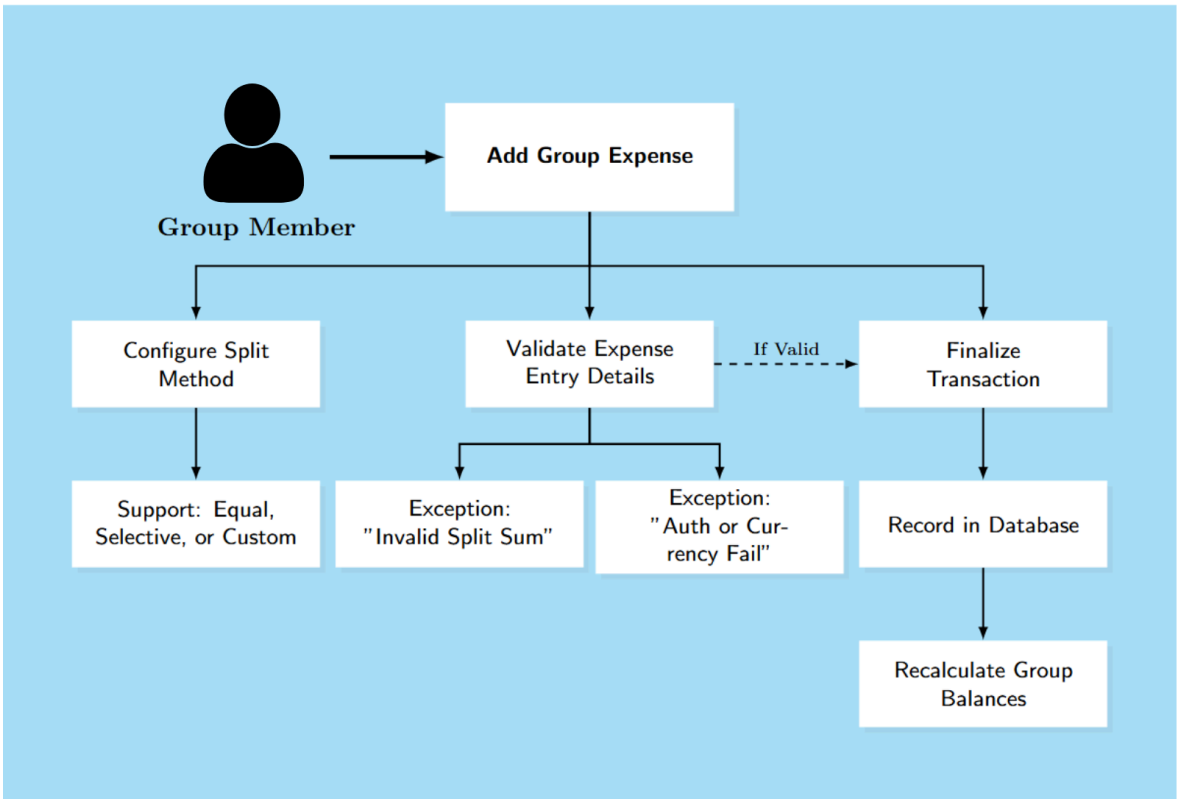
| | |
|---|---|
| | email address or phone number.<br>● OTP Login Failure:<br>1. If the user selects OTP-based login and enters an incorrect OTP, the system shall display: *"Incorrect OTP entered. Please try again."*<br>2. If the OTP expires, the system shall display: *"OTP expired. Please request a new OTP."* |
| Includes | **Use Case #1: User Registration (U1)** (for unregistered users redirected to registration) |
| Notes/Issues | ● The system shall support multiple authentication methods, including password-based login, Google Sign-In, and OTP-based login.<br>● OTP-based login and password recovery shall follow the same security constraints regarding OTP validity duration.<br>● Successful authentication is required before granting access to any protected application features. |

### 3.3.3 Use Case #3: Create or Join Group (U3)

| | |
|---|---|
| Authors | This use case was written by Rushabh Pandya and Sanket Agarwal. |
| Purpose | To allow a logged-in user to create a new group or join an existing group using a valid invite link. |
| Requirements Traceability | FR-5, FR-6 |
| Priority | High - Core functionality for group management |
| Preconditions | The user must be logged in to the application. |
| Post conditions | ● The user becomes a member of the selected group.<br>● The user gains access to group-related data, including expenses, balances, photos, and documents. |

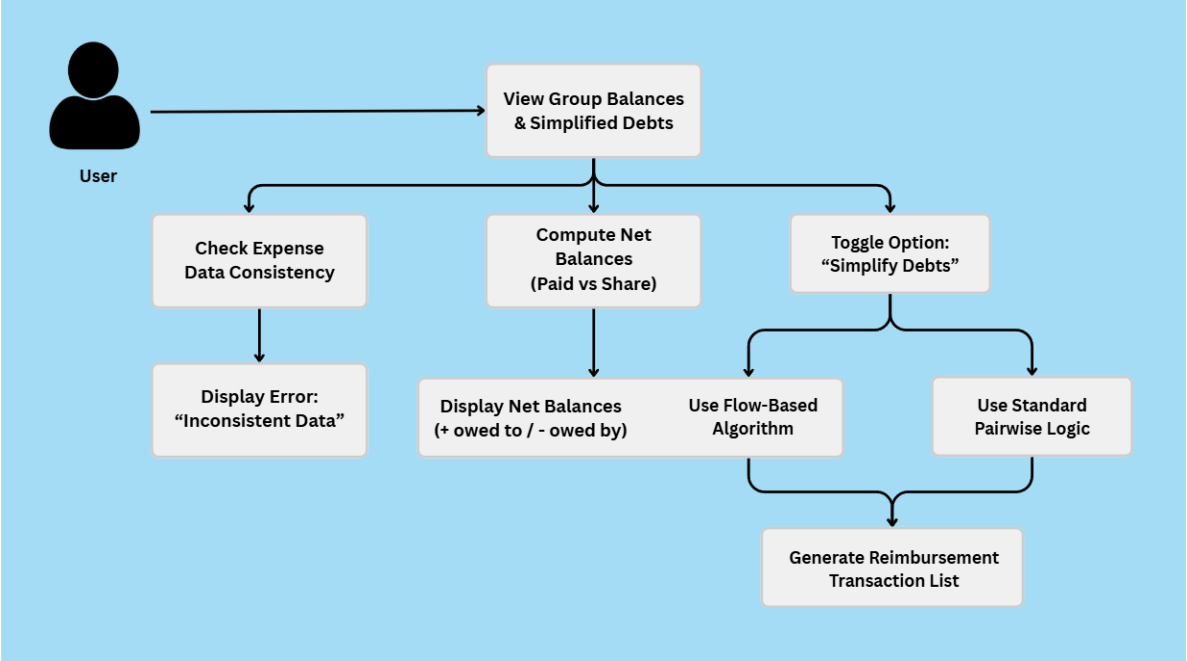
| | |
|---|---|
| Actors | User, System |
| Exceptions | • Invalid or Expired Invite Link: The system shall display an error indicating that the invite link is invalid or has expired.<br>• Group Not Found: The system shall display an error if the group associated with the invite link no longer exists.<br>• Duplicate Membership: The system shall prevent the user from joining a group they are already a member of and display an appropriate message. |
| Includes | None |
| Notes/Issues | Late-joining members shall be able to view all existing group data, including previously recorded expenses, balances, photos, and documents. |

### 3.3.4 Use Case #4: Add Group Expense (U4)



| | |
|---|---|
| Authors | This use case was written by Mohd Sarim Ali and Ronit Kumar. |
| Purpose | To allow a group member to record a shared expense within a group. |
| Requirements Traceability | FR8 |
| Priority | High - Core functionality |
| Preconditions | The user must be a member of the trip group. |
| Post conditions | ● The expense is successfully recorded in the system.<br>● Group balances are updated to reflect the newly added expense. |
| Actors | User, System, Database |
| Exceptions | ● Invalid Split Configuration: If the split amounts do not sum to the total expense amount, the system shall reject the entry and display an error.<br>● Currency Conversion Failure: If currency conversion cannot be completed, the system shall notify the user and prevent expense submission.<br>● Unauthorized Action: If a user attempts to log an expense for a non- member of the group, the system shall deny the action. |
| Includes | None |
| Notes/Issues | ● Expense entries shall be reflected immediately in the group's expense history and balance overview.<br>● The system supports multiple splitting methods: equal split, selective split, and custom split amounts.<br>● Any modification or removal of recorded expenses may trigger recomputation of group balances. |

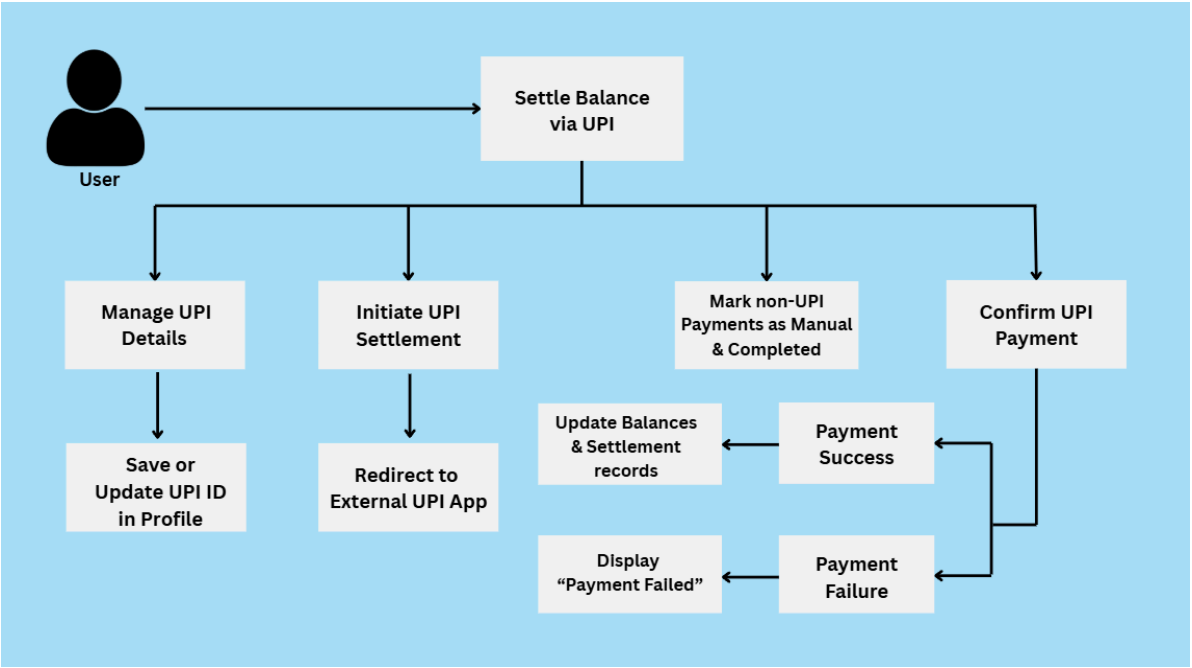
### 3.3.5 Use Case #5: View Group Balances & Simplified Debts (U5)




| | |
|---|---|
| Authors | This use case was written by Pranjali Singh and Raghav Madan. |
| Purpose | To allow users to view net group balances and the set of reimbursement transactions required to settle outstanding amounts. |
| Requirements Traceability | FR-9 |
| Priority | High - Core computation feature |
| Preconditions | At least one expense exists in the group. |
| Post conditions | • Net balances for all group members are computed and displayed.<br>• A list of reimbursement transactions is generated based on the selected settlement mode. |
| Actors | User, System |

| | |
|---|---|
| Exceptions | Inconsistent Expense Data: If expense records are incomplete or inconsistent, the system shall notify the user and prevent settlement computation. |
| Includes | None |
| Notes/Issues | • The system computes reimbursement transactions using flow-based techniques to ensure correctness and efficiency.<br>• Users may toggle the "Simplify Debts" option to reduce the total number of reimbursement transactions, which may alter original debtor-creditor relationships.<br>• A positive balance indicates money owed to the member; a negative balance indicates money owed by the member. |

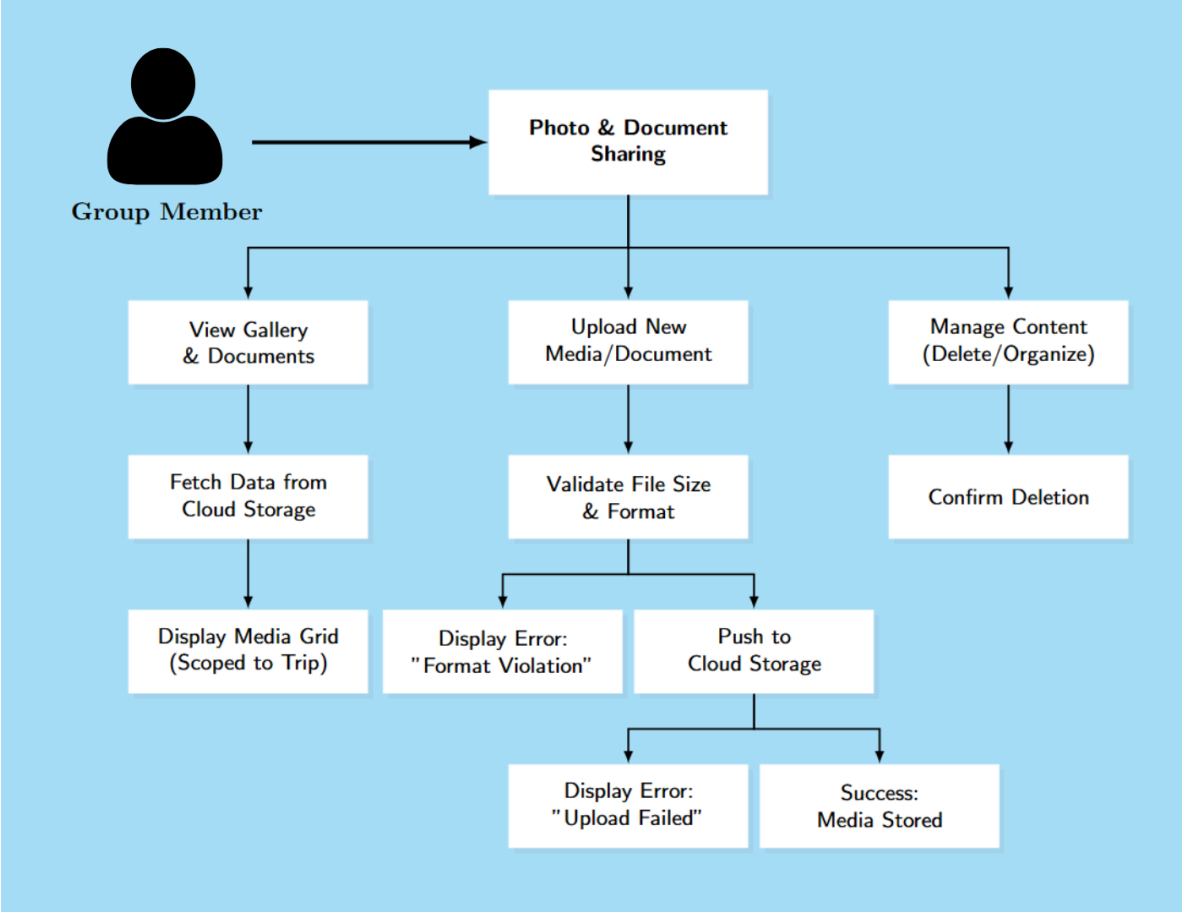
### 3.3.6 Use Case #6: Settle Balance via UPI (U6)




| | |
|---|---|
| Authors | This use case was written by Pranjali Singh and Mohd Sarim Ali. |
| Purpose | To allow users to settle outstanding balances using UPI-based payment redirection or manual confirmation. |
| Requirements Traceability | FR-10 |

| | |
|---|---|
| Priority | High - Core functionality |
| Preconditions | • The user must have a non-zero outstanding balance.<br>• The payee must have a valid UPI ID saved in their profile. |
| Post conditions | The settlement is recorded in the system and group balances are updated accordingly. |
| Actors | User, Payment Gateway, System |
| Exceptions | • Payment Failure: If the payment is unsuccessful, the system shall not update balances and shall notify the user.<br>• Payment Cancelled: If the user cancels the payment, no settlement is recorded.<br>• UPI Application Unavailable: If no compatible UPI application is installed, the system shall notify the user. |
| Includes | None |
| Notes/Issues | • Settlement updates require explicit user confirmation and are not assumed automatically.<br>• Manual settlements may be recorded when payments occur outside the UPI flow. |

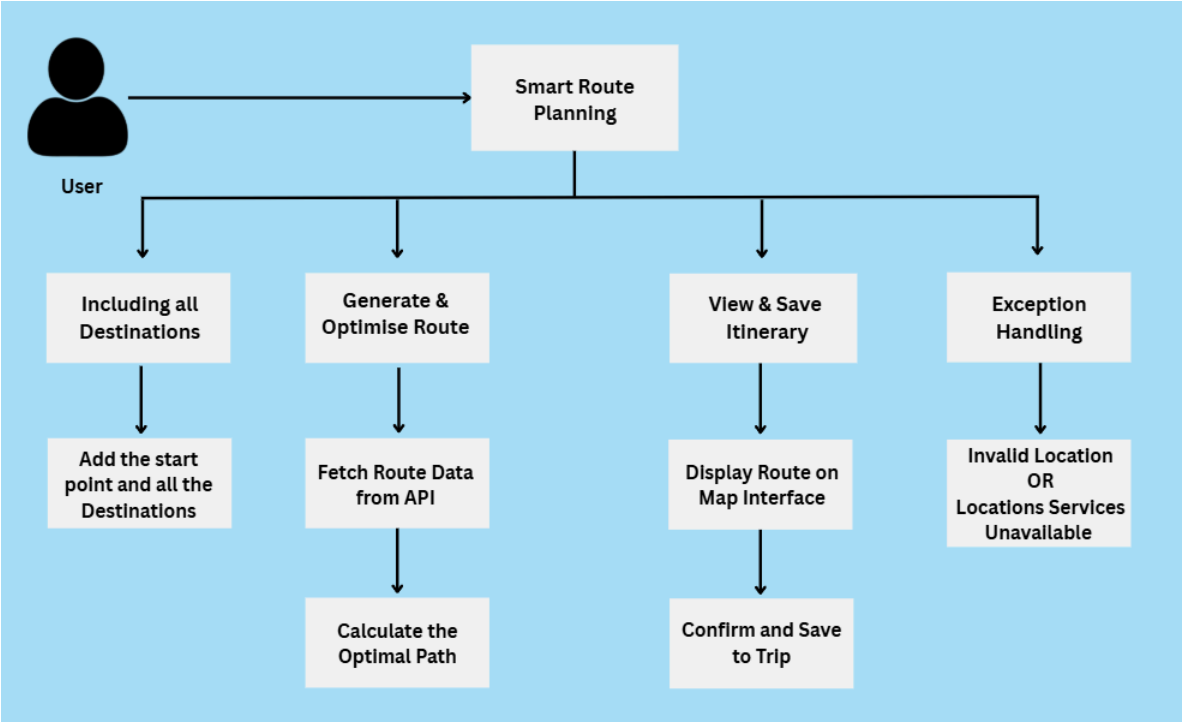
### 3.3.7 Use Case #7: Photo & Document Sharing (U7)




| | |
|---|---|
| Authors | This use case was written by Ayush Jha and Rushabh Pandya. |
| Purpose | To allow group members to upload, view, and manage photos and documents associated with a trip. |
| Requirements Traceability | FR-11 |
| Priority | Medium - Important feature for trip documentation |
| Preconditions | The user must be a member of the trip group. |
| Post conditions | Media files are successfully stored and accessible to group members. |

| | |
|---|---|
| Actors | User, System, Cloud Storage |
| Exceptions | • File Size or Format Violation: If the uploaded file exceeds allowed limits or is unsupported, the system shall reject the upload.<br>• Upload Failure: If media upload fails due to connectivity or storage issues, the system shall notify the user. |
| Includes | None |
| Notes/Issues | • Access to photos and documents is restricted to group members only.<br>• Media visibility is scoped to the corresponding trip. |

### 3.3.8 Use Case #8: Smart Route Planning (U8)




| | |
|---|---|
| Authors | This use case was written by Raghav Madan and Khushi Khandelwal. |
| Purpose | To generate an optimized visiting order for selected |

| | |
|---|---|
| | locations within a trip. |
| Requirements Traceability | FR-12 |
| Priority | Medium - Useful feature for trip planning |
| Preconditions | The user must select a starting location, an ending location, and at least one additional location to visit. |
| Post conditions | An optimized visiting order covering all selected locations is generated and displayed. |
| Actors | User, System, Location Service |
| Exceptions | • Location Services Unavailable: If distance or location data cannot be retrieved, the system shall notify the user.<br>• Invalid Location Input: If selected locations are invalid or unreachable, the system shall prevent route generation. |
| Includes | None |
| Notes/Issues | • Exact optimality is not guaranteed for large input sets.<br>• The system uses external location services to calculate distances between locations. |

# 4 Other Non-functional Requirements

## 4.1 Performance Requirements

- **Concurrency:** The system shall support at least 5,000 concurrent users without service disruption or data inconsistency.
- **Responsiveness:** The average response time shall be below 3 seconds during application startup.
  The average response time for standard user operations during normal usage shall be under 500 milliseconds.
- **Resource Efficiency:** The system shall optimize the use of device storage, memory, network bandwidth, and battery life by limiting background processing, caching only necessary data and compressing images and documents where applicable thereby minimizing unnecessary data usage.

## 4.2 Safety and Security Requirements

- **Authentication and Authorization:** The system shall implement secure authentication mechanisms using password-based login (email/phone), OTP-based verification, and optional Google Sign-In to prevent unauthorized access.
- **Secure Data Transmission:** All data exchanged between the client application and backend services shall be transmitted securely and reliably to prevent data breaches
- **Secure Data Storage:** Sensitive user data (e.g., tokens, UPI ID) shall be stored using platform-provided secure storage mechanisms.
- **Access Control:** Only authenticated users who are members of a group shall be allowed to view, add, or modify that group's expenses, balances, photos, and documents.
- **Data Backup and Recovery:** The system shall maintain periodic backups of application data and media metadata to prevent data loss.
- **Input Validation:** The system shall validate user inputs (amounts, currency, UPI ID format, group link/join name) to prevent invalid records and injection-style attacks.
- **Minimum Platform Standards:** The application shall support a minimum set of operating system versions, including:
  - Android 8+
  - iOS 13+
- **Notification Safety:** Settlement requests/reminders and group activity notifications shall not expose sensitive financial details on the device lock screen unless explicitly enabled by the user.
- **Password Strength Enforcement** : The system shall enforce strong passwords by preventing users from setting passwords that meet any of the following criteria:
  - Length is less than 8 characters.
  - Contains only lowercase or uppercase alphabets.
  - Uses commonly known phrases, examples of which will be stored in a database for reference.
- **Password Recovery Security:** Password recovery shall be protected using OTP-based verification to minimize the risk of unauthorized account access.

## 4.3 Software Quality Attributes

### 4.3.1 Maintainability

- The system architecture and codebase shall be modular to reduce maintenance effort and improve extensibility.
- Fixing a critical defect (e.g., security flaw or payment/settlement inconsistency), including testing and documentation updates, shall not exceed two working days.
- Adding a minor feature(e.g., a new group setting ) or enhancement, including testing and documentation updates, shall not exceed one person-week.

### 4.3.2 Availability

- In case of a backend failure, the system shall be restored within 2 hours to minimize service disruption.
- The application shall remain functional under peak activity (e.g., during trips when many expenses are added) without major performance degradation.
- The system shall support limited offline access, allowing previously synchronized group data to be viewed and updated data to be synchronized once network connectivity is restored, where feasible.

### 4.3.3 Reliability

- The system shall ensure consistent and accurate computation of expenses, balances, and settlement updates across all group members.
- The system shall undergo unit testing, integration testing, regression testing, and load testing prior to deployment.
- The system shall prevent duplicate or conflicting settlement updates (e.g., double “mark as paid”) through appropriate server-side validation and transaction handling.

### 4.3.4 Portability

- The front end shall be developed using Flutter to ensure compatibility across Android and iOS devices.
- The application shall support multiple screen sizes (phones and tablets) and different resolutions with consistent user experience.

### 4.3.5 Usability

- The application shall provide an intuitive and consistent user interface suitable for first-time users without requiring external guidance.

- Common actions such as adding expenses, viewing balances, and settling payments shall be achievable within a minimal number of interactions.

- The system shall display clear error messages and actionable feedback for invalid inputs, failed operations, or network issues.

### 4.3.6 Scalability

- The system architecture shall support growth in the number of users, groups, expenses, and stored media without requiring fundamental redesign.
- The application shall maintain acceptable performance as group size and historical trip data increase.

### 4.3.7 Offline Tolerance and Network Resilience

- The system shall allow users to view previously synchronized group data (expenses, balances, photos metadata) in offline mode.
- Any actions performed while offline shall be queued locally and synchronized with the backend once network connectivity is restored

### 4.3.8 Auditability and Transparency

- All expense additions, edits, deletions, and settlement actions shall be logged.
- The system shall ensure that historical expense and settlement records remain immutable once settled, except for clearly tracked administrative corrections.
- Users shall be able to trace how their current balance was computed from past expenses and settlements.

### 4.3.9 Accessibility and Compatibility

- The application shall support a wide range of screen sizes and orientations while maintaining consistent UI behavior.
- The system shall ensure sufficient color contrast and readable text sizes to support accessibility.
- The application shall remain usable with standard mobile accessibility tools provided by the operating system.

# 5 Other Requirements

## 5.1 Authentication and Verification Requirements

- The system shall require OTP-based verification for critical authentication actions, including user registration, password reset, and changes to registered email address or phone number.
- OTPs shall be time-bound and expire after a configurable duration (e.g., 10 minutes) to prevent misuse.
- Google Sign-In authentication shall comply with the security policies and usage guidelines defined by the authentication provider.

## 5.2 Legal, Privacy, and Compliance Requirements

- The system shall collect, store, and process user data in accordance with applicable data protection laws, including but not limited to the Digital Personal Data Protection Act (DPDPA).
- User consent shall be obtained before accessing sensitive device permissions such as location, storage, or notifications.
- Financial identifiers such as UPI IDs shall be used only for settlement purposes and shall not be shared with other users without explicit action by the user.
- Uploaded photos and documents shall be accessible only to members of the corresponding trip group.

## 5.3 Payment and Financial Safety Requirements

- The system shall not directly process or store UPI transaction credentials and shall rely only on UPI intent-based redirection to authorized payment applications.
- The system shall record settlement confirmations only after explicit user acknowledgment (e.g., “Did the payment go through?”).
- The system shall maintain an immutable audit log of settlement actions (UPI-based or manual) for transparency and dispute resolution.
- The system shall prevent duplicate settlement confirmations using server-side validation.

## 5.4 Database and Backup Requirements

- All critical data, including users, groups, expenses, balances, and settlements, shall be stored in a persistent backend database.

- The system shall perform periodic backups of structured data and file metadata to prevent data loss.

- Database operations related to expense logging and settlement updates shall be atomic to ensure consistency.

- Deleted user accounts shall not remove historical expense data from groups to preserve financial correctness.

## 5.5 Third-Party Service Dependency Requirements

- The system depends on external services such as OTP providers, map/location APIs, UPI-enabled payment applications, and cloud storage services.

- Temporary unavailability of third-party services shall not cause permanent data loss and shall be handled gracefully with appropriate user notifications.

- The system shall degrade non-critical features (e.g., route planning) gracefully if external services are unavailable.

## 5.6 Data Retention and Cleanup Requirements

- Trip data, including expenses and settlements, shall be retained for reference even after the trip is marked as completed.

- Users shall be allowed to manually delete photos and documents they upload, subject to group permissions.

- Cached local data shall be cleared automatically upon logout or account deletion to protect user privacy.

# Appendix A – Data Dictionary

| Element Name | Description | Attributes | Operations | Relationships |
|---|---|---|---|---|
| User | Registered user of Travelly | userId, name, email, phone, passwordHash, UPI_ID, theme, notifications | register, login, logout, updateProfile, deleteAccount | Can belong to multiple groups |
| Group | Travel group created for a trip | groupId, title, photo, currency, inviteLink, createdBy | createGroup, joinGroup, addParticipant, leaveGroup | Contains multiple users and expenses |
| Participant | Member within a group | participantId, name, userId(optional) | addParticipant, removeParticipant | Belongs to a group |
| Expense | Recorded spending in a group | expenseId, title, amount, currency, paidBy, date | addExpense, editExpense, deleteExpense | Linked to group and participants |
| ExpenseSplit | Distribution of an expense | splitId, participantId, amount | calculateSplit | Associated with expense |
| Balance | Net amount owed or to receive | userId, groupId, balanceAmount | calculateBalance, updateBalance | Derived from expenses |
| Settlement | Payment done to clear balance | settlementId, fromUser, toUser, amount, method, status | payUPI, markAsPaid | Linked to balances |
| Media | Photos and documents of trip | mediaId, type, fileURL, uploadedBy, timestamp | uploadMedia, viewMedia, deleteMedia | Associated with group |
| RoutePlan | Optimized travel route | routeId, startLocation, endLocation, locations[] | generateRoute | Used within a group |
| Location | Represents a place used in route planning | locationId, latitude, longitude, placeName | fetchLocation(), validateLocation() | Used in RoutePlan |

| | | | | |
|---|---|---|---|---|
| Notification | System-generated alerts for users | notificationId, type (Payment/Reminder/GroupUpdate), message, isRead | sendNotification(), markRead() | Triggered by settlements and group updates |
| AppSettings | Stores global user preferences | themeMode, notificationEnabled | updateSettings() | Linked to User |
| Currency | Defines currency used in group expenses | currencyCode (INR, USD, etc.), symbol | convertCurrency() | Used in Expense and Settlement |
| SystemDatabase | Backend persistent data storage | usersTable, groupsTable, expensesTable, settlementsTable | store(), update(), delete(), retrieve() | Central dependency for all modules |
| LocationServiceAPI | External service for maps and distance calculation | distanceMatrix, placeSuggestions | getDistance(), getRouteData() | Used in route optimization |
| PaymentGateway (UPI Apps) | External payment applications for settlements | GPay, PhonePe, Paytm | redirectPayment(), returnStatus() | Used during balance settlements |

# Appendix B - Group Log

| Date | Timing | Duration | Summary of the meeting |
|---|---|---|---|
| 08/01/2026 | 7 pm - 8pm | 1hr | • Initial discussion on project ideas from multiple domains including travel planning, expense management, and productivity tools. |
| 09/01/2026 | 7pm - 9pm | 2hr | • Expanded discussion on shortlisted ideas with focus on feasibility, scope, and relevance to student use cases. |
| 10/01/2026 | 11am - 12:30pm | 1.5hr | • Evaluated technical and practical feasibility of chosen idea; finalized “Travelly” as the project name. |
| 15/01/2026 | 7pm - 9pm | 2hr | • Detailed discussion on functional and system requirements keeping in mind the end users; identified core features such as group creation, expense management, and settlements. |
| 18/01/2026 | 2pm - 5pm | 3hr | • Finalized SRS timeline and milestones; divided documentation and feature responsibilities among team members. |
| 20/01/2026 | 7pm - 9:30pm | 2.5hr | • Worked on requirement documentation; clarified doubts related to UI expectations after discussion with the TA. |
| 22/01/2026 | 10am - 12pm<br>8pm - 11pm | 5hr | • Reviewed and consolidated SRS sections prepared by different members; resolved inconsistencies and synchronized content. |
| 23/01/2026 | 7pm - 9pm | 2hr | • Final review and proofreading of the SRS document; incorporated final corrections and finalized submission. |